VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



NGÔ LÂM TIẾN — 524H0195
NGUYỄN CÔNG THÀNH — 524H0191
NGUYỄN LÊ NHẬT TÂM — 524H0186

MIDTERM REPORT

INTRODUCTION TO MACHINE LEARNING

Optimization, Classification and Generalization

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



NGÔ LÂM TIẾN — 524H0195
NGUYỄN CÔNG THÀNH — 524H0191
NGUYỄN LÊ NHẬT TÂM — 524H0186

MIDTERM REPORT

INTRODUCTION TO MACHINE LEARNING

Advised by

Faculty of Information Technology

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our instructor and the Faculty of Information Technology at Ton Duc Thang University for their dedicated instruction, insightful feedback, and continuous guidance throughout the Introduction to Machine Learning course and this midterm project. Through this assignment, we have developed a rigorous understanding of optimization foundations from first principles, sound empirical benchmarking protocols, and essential principles for preventing data leakage and controlling model capacity.

We also extend our appreciation to our fellow students and academic peers for constructive discussions and collaborative sharing of machine learning insights. We are deeply honored and privileged to conduct this research under the academic standards of Ton Duc Thang University.

Ho Chi Minh city, October 2026.

Authors

(Signature and full name)

Ngô Lâm Tiến **Nguyễn Công Thành** **Nguyễn Lê Nhật Tâm**

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by our instructor; The research derivations, algorithmic implementations, and empirical results contained herein are our original work and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected and processed by our group from documented sources, which are clearly stated in the reference section.

In addition, the project also uses comments, assessments as well as standard algorithms and baseline data of other authors and organizations with clear citations and annotated sources.

If any violation occurs, we will take full responsibility for the content of our project. Ton Duc Thang University is not responsible for any copyright or intellectual property infringements during our implementation process (if any).

Ho Chi Minh city, October 2026

Authors

(Signature and full name)

Ngô Lâm Tiến **Nguyễn Công Thành** **Nguyễn Lê Nhật Tâm**

TABLE OF CONTENT

CHAPTER 1. INTRODUCTION **1**

1.1 Objectives and experimental overview **1**

1.2 Reading map and experimental boundaries **1**

CHAPTER 2. THEORETICAL BASIS **2**

2.1 Linear Regression and its gradient **2**

2.2 Batch Gradient Descent and verification **3**

2.3 Linear Regression experiment **4**

2.4 Logistic Regression and cross-entropy **5**

2.5 Stable scratch implementation **6**

2.6 Logistic Regression experiment **7**

CHAPTER 3. DESIGNING AND IMPLEMENTING **8**

3.1 OpenML dataset and provenance **8**

3.2 Exploratory analysis and cleaning **9**

3.3 Preprocessing without leakage **10**

3.4 Model comparison protocol **11**

3.5 Cross-validation and test comparison **12**

3.6 Selected model and generalization **13**

CHAPTER 4. OVERFITTING ANALYSIS AND MITIGATION **14**

4.1 Overfitting across model families **14**

4.2 Empirical mitigation and trade-offs **15**

4.3 Discussion and remaining limitations **16**

REFERENCES **17**

APPENDIX. REPRODUCIBILITY AND VERIFICATION **18**

CHAPTER 1. INTRODUCTION

1.1 Objectives and experimental overview

This report addresses the two equally weighted questions in the original Middle Exam. Question 1 derives and trains Linear Regression and binary Logistic Regression from first principles. Question 2 studies a classification dataset downloaded from OpenML, with preparation, model comparison and overfitting analysis.

On synthetic held-out data, Linear Regression achieves RMSE 0.6462 and R^2 0.9655; Logistic Regression achieves accuracy 0.8833 and F1 0.8772. On German Credit, five-fold development CV selects HistGradientBoosting from 12 predefined learning algorithms. Its independent test accuracy is 0.7550 and macro-F1 is 0.6874.

Results are generated by executable scripts, not transcribed from the previous submission. Analytical gradients are tested on multiple shapes and seeds, then checked against independent numerical and library references. Official raw data, metadata, checksums, split indices, fold scores and test predictions are retained.

Reading map

Topic	Location
Linear Regression theory, implementation and evidence	Sections 2-4; Q1_Linear_Regression.ipynb
Logistic Regression and probability evaluation	Sections 5-7; Q1_Logistic_Regression.ipynb
OpenML source, EDA and preprocessing	Sections 8-10; src/data.py
Classifier comparison and final evaluation	Sections 11-13; Q2_Classification.ipynb
Overfitting, mitigation and limitations	Sections 14-16
References and reproduction	Sections 17-18; README.md; verification/

Experimental boundaries

Synthetic Q1 tasks illustrate optimization under controlled conditions. The historical credit dataset is an educational benchmark. Neither establishes deployment validity or a guaranteed grade. Model selection uses development CV only; all candidate test results are descriptive and do not change the selected procedure.

CHAPTER 2. THEORETICAL BASIS

2.1 Linear Regression and its gradient

Regression predicts a continuous response; classification predicts a discrete label. With m observations and d features, X is an m by d matrix, w is a d-dimensional vector, b is a scalar intercept and y is an m-dimensional response. The affine prediction can represent a nonzero response when all features are zero.

$$\hat{y}_i = \mathbf{x}_i^T \mathbf{w} + b, \quad e_i = \hat{y}_i - y_i$$

$$J(\mathbf{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} e_i^2$$

$$\frac{\partial J}{\partial w_j} = \frac{1}{2m} \sum_i 2 e_i \frac{\partial e_i}{\partial w_j} = \frac{1}{m} \sum_i e_i x_{ij}$$

$$\frac{\partial J}{\partial b} = \frac{1}{2m} \sum_i 2 e_i \cdot 1 = \frac{1}{m} \sum_i e_i$$

$$\nabla_{\mathbf{w}} J = \frac{X^T (X\mathbf{w} + b\mathbf{1} - \mathbf{y})}{m}$$

The chain rule connects the squared residual to each parameter through the prediction. The factor of one half cancels the derivative of the square. It rescales the loss, but does not change the least-squares minimizer. Reported MSE is twice this training objective.

Why squared error?

MSE measures average squared prediction error and penalizes large residuals. Under independent Gaussian errors with constant variance, minimizing it also maximizes the likelihood of the linear mean model. This assumption is appropriate for the synthetic data-generating process here; it is not required merely to calculate an MSE.

The objective is convex because its Hessian is positive semidefinite. Convexity does not make every learning rate safe: steps that are too large can diverge. Rank-deficient inputs can admit multiple least-squares solutions; this experiment has independent features.

2.2 Batch Gradient Descent and verification

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} J(\mathbf{w}_t, b_t)$$

$$b_{t+1} = b_t - \eta \frac{\partial J(\mathbf{w}_t, b_t)}{\partial b}$$

Both parameters are updated from gradients computed at the same current parameters. Full-batch training uses every training record in each update. The implementation initializes weights and intercept to zero, so initialization is deterministic without a random draw.

Step	Implemented operation
Validate	Require finite, nonempty (m,d) features and matching (m,) or (m,1) target; require positive learning rate and integer iterations.
Initialize	Set w=0 and b=0; record the initial loss.
Iterate	Compute loss and both gradients in NumPy; reject nonfinite values; apply simultaneous gradient updates.
Monitor	Record every post-update loss and the final gradient norm; use fixed training iterations.
Predict	Validate feature count, then compute Xw+b.

Independent finite differences

$$g_j^{\mathrm{FD}} = \frac{J(\theta + \varepsilon \mathbf{e}_j) - J(\theta - \varepsilon \mathbf{e}_j)}{2\varepsilon}, \quad \varepsilon = 10^{-6}$$

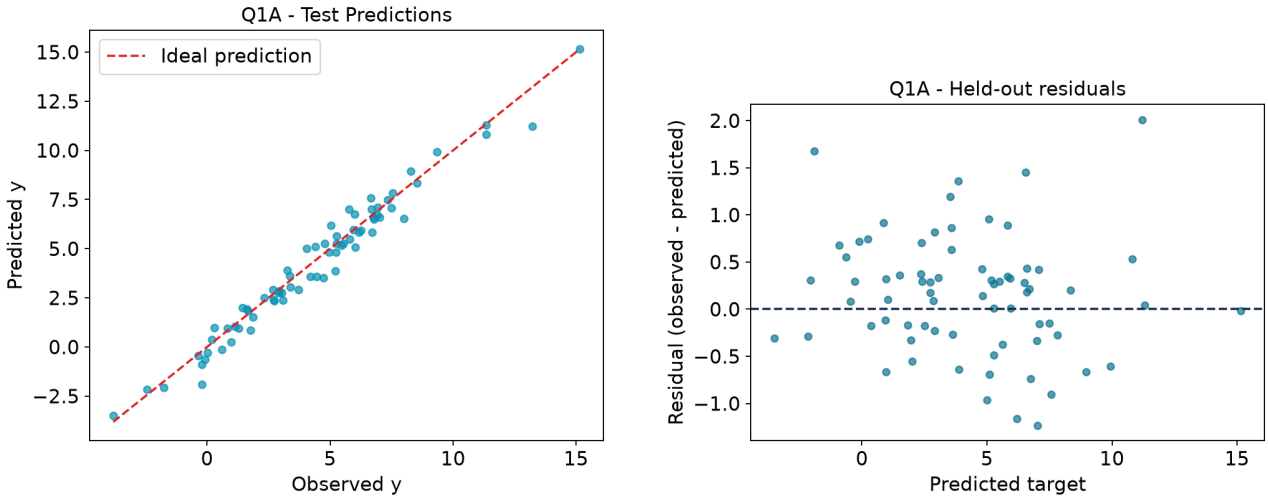
The gradient test covers four seeds and four (m,d) shapes: (1,1), (7,1), (9,3), (5,6), for both models: 32 comparisons. Every maximum absolute error must be below 10^{-7}. Moderate random parameters avoid saturation and excessively ill-scaled differences. Absolute tolerance is specified rather than an unstable relative error near zero.

Additional checks verify an explicit one-step update, the intercept, predictions, decreasing loss, extreme logits and deterministic fits. No scikit-learn optimizer trains the submitted scratch models. Library fits appear only as separately labeled validation benchmarks.

2.3 Linear Regression experiment

Seed 42 generates 360 standard-normal feature pairs and y=4+3x1-2x2+Gaussian noise with standard deviation 0.65. A fixed 80/20 split gives 288 training and 72 test records. StandardScaler learns its mean and variance from training features only. Gradient Descent uses learning rate 0.08 and 1,200 updates.

Measurement	Executed result
Training objective, initial / final	13.728956 / 0.209771
MSE / RMSE	0.417594 / 0.646215
MAE / R^2	0.504040 / 0.965492
Final gradient norm	3.639e-15
Gradient check maximum absolute difference	1.200e-09
Maximum prediction difference from least squares	7.105e-15



Figures 2-3. Held-out predictions and residuals use the same 72 test records. The diagonal is ideal prediction; the horizontal residual line is zero.

RMSE 0.646 is close to the injected noise standard deviation, while residual scatter shows the unavoidable random component. R^2 compares squared error with the test-mean baseline; it does not measure causal explanation. The high value is expected for this deliberately linear problem.

The independent reference uses sklearn LinearRegression least squares. Its predictions agree within 10^{-8}, supporting successful optimization of this objective. This validation does not replace the from-scratch fitting loop.

2.4 Logistic Regression and cross-entropy

Binary Logistic Regression models P(y=1|x). A linear logit z is transformed to probability p by the sigmoid. The default decision predicts label 1 when p is at least 0.5. Unlike Linear Regression, the output is constrained to a probability, while the decision boundary remains linear in the transformed features.

$$z_i = \mathbf{x}_i^T \mathbf{w} + b, \quad p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$$

$$\ell_i = -y_i \log p_i - (1-y_i)\log(1-p_i), \quad J = \frac{1}{m}\sum_i \ell_i$$

$$\frac{\partial \ell_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i} = \frac{p_i - y_i}{p_i(1-p_i)}$$

$$\frac{\partial p_i}{\partial z_i} = p_i(1-p_i), \quad \frac{\partial \ell_i}{\partial z_i} = p_i - y_i$$

$$\nabla_{\mathbf{w}} J = \frac{X^T(\mathbf{p}-\mathbf{y})}{m}, \quad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_i (p_i - y_i)$$

For finite real logits, 0<p<1 mathematically, so the cancellation in the chain rule is valid. The last line differentiates the average objective; the preceding lines differentiate one observation, avoiding an omitted factor of 1/m.

Why binary cross-entropy?

A binary response has Bernoulli likelihood p^y(1-p)^(1-y). Taking its negative logarithm gives the loss above. Confident incorrect probabilities incur a large penalty. BCE is a probability estimation objective; neither a low BCE nor a high ROC-AUC proves calibrated probabilities on a new population.

2.5 Stable scratch implementation

$$\ell_i = \log(1 + e^{z_i}) - y_i z_i$$

$$J = \text{mean}(\text{logaddexp}(0, z) - yz)$$

The computation uses NumPy logaddexp rather than log of a probability that can round to zero. Sigmoid evaluates 1/(1+exp(-z)) for nonnegative logits and exp(z)/(1+exp(z)) for negative logits. Neither branch exponentiates a large positive argument.

Component	Exact behavior
Parameter arrays	weights_: shape (d,); bias_: scalar; predict_proba returns shape (m,).
Targets	Binary values 0/1 only; nonfinite and mismatched inputs rejected.
Training	Learning rate 0.13, 2,400 full-batch updates, zero initialization.
Numerical monitoring	Nonfinite loss/gradients raise errors; loss history includes initial and final updated parameters.
Threshold	0.5 for evaluation; thresholds outside [0,1] rejected.
Extreme-input test	Logits -1000, 0, +1000 produce probabilities 0, 0.5, 1 without overflow; wrong labels give finite BCE 1000.

Convergence evidence

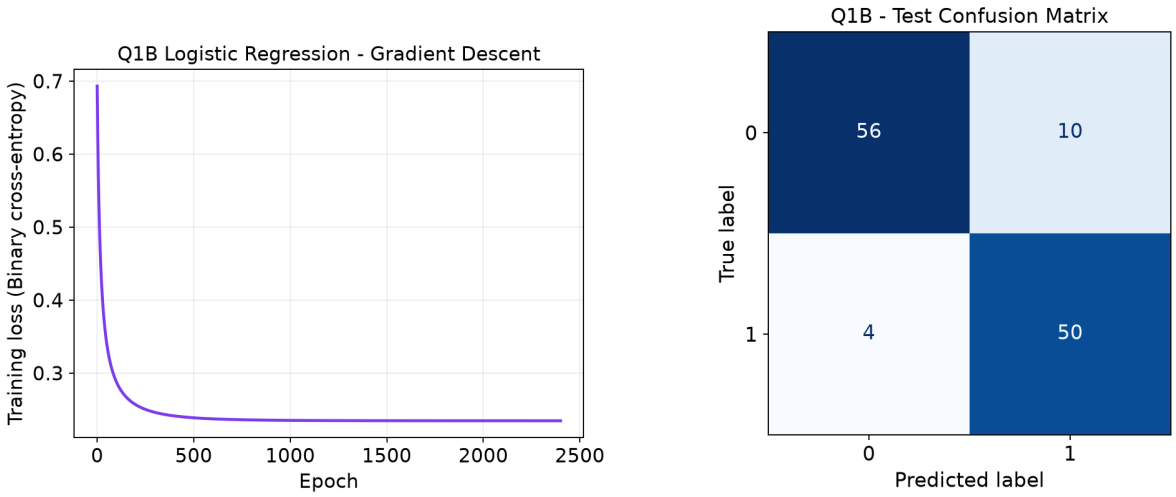
The final gradient norm is 3.265e-04. Fixed-step training approaches the optimum but is not claimed to be exactly stationary. An independent unregularized sklearn LBFGS fit reaches training loss 0.23468148; the scratch loss is 0.23468563. Maximum probability disagreement is 0.001991, below the predeclared 0.005 tolerance.

A reproducible synthetic dataset includes label noise and overlapping classes. Such data avoid the claim that finite coefficients must exist for every possible perfectly separable dataset. Unregularized Logistic Regression can increase coefficient magnitude without reaching a finite optimum under complete separation.

2.6 Logistic Regression experiment

make_classification uses 600 samples, five features, four informative features, no redundant features, weights 0.55/0.45, class separation 1.2, label-flip probability 0.04 and seed 27. The split uses seed 42 and stratification, giving 480 training and 120 test records. Class 1 is positive. Scaling is learned only from the training records.

Measurement	Executed result
Initial / final training BCE	0.693147 / 0.234686
Test accuracy	0.883333
Positive precision / recall	0.833333 / 0.925926
Positive F1 / ROC-AUC	0.877193 / 0.925084
Gradient check maximum absolute difference	3.606e-11
Confusion matrix [TN, FP; FN, TP]	[[56, 10], [4, 50]]



Figures 4-5. BCE over 2,400 updates and confusion matrix at threshold 0.5; outputs trace to q1_arrays.npz.

There are 10 false positives and 4 false negatives. Recall is higher than precision, so the chosen threshold captures more positive cases while accepting some false alarms. ROC-AUC uses probabilities, assessing ranking across thresholds rather than classification at 0.5.

MSE, RMSE and MAE are regression metrics; precision, recall, F1 and confusion matrices reflect classification decisions. Evaluation therefore matches each Q1 problem rather than applying the same metric to both.

CHAPTER 3. DESIGNING AND IMPLEMENTING

3.1 OpenML dataset and provenance

Question 2 uses credit-g, a historical German Credit risk classification dataset. Its seven numerical and thirteen categorical predictors provide a substantive setting for encoding, scaling and generalization analysis. It replaces the previous sklearn Iris mirror because official OpenML retrieval was not established for that artifact.

Provenance item	Recorded evidence
Official identity	OpenML ID 31, version 1, name credit-g [1]
Rows / predictors / target	1,000 / 20 / class
Label distribution	700 good; 300 bad
Official download	https://openml.org/data/v1/download/31/credit-g.arff
Retrieval time UTC	2026-10-09T13:00:45.773055+00:00
Retrieval method	OpenML v1 JSON API then API-provided ARFF URL via urllib.request; raw bytes retained.
MD5 matches OpenML	9a475053fed0c26ee95cd4525e50074c
Raw SHA-256	768be84ef749110d5fcac109a6541f2d f190e8c6987e1dc432c0a4f29d0642f5
Raw size	162291 bytes
Redistribution	OpenML metadata labels Public; originating UCI dataset is CC BY 4.0, attributed to H. Hofmann (1994) [2].

Verification status: VERIFIED

The downloaded MD5 exactly matches the checksum in the official JSON response. A stronger local SHA-256 identifies the retained artifact. The parser validates dataset dimensions, target classes, numeric finiteness and nominal domains. The offline experiment reuses these exact raw bytes and verifies both recorded hashes before parsing.

The cached file is an official downloaded artifact, not a reconstruction from another library. Reacquisition compares bytes and rejects unexpected changes. The metadata response and acquisition log are supplied separately from the raw ARFF.

3.2 Exploratory analysis and cleaning

Raw schema validation finds zero missing cells and zero exact duplicate records; no rows are deleted and no artificial problems are injected. The 70/30 good/bad class imbalance makes majority accuracy potentially misleading. Exploratory plots and descriptive statistics use the 800-record development partition, keeping test feature distributions out of modelling decisions.

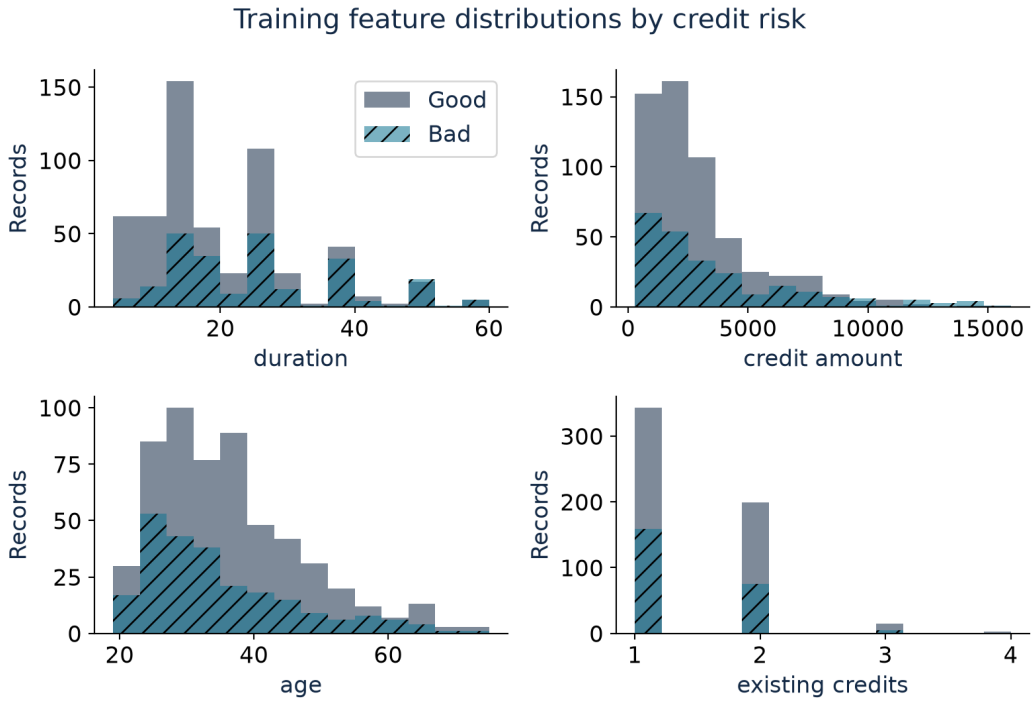


Figure 5. Distributions of four development predictors, with risk labels distinguished by legend and hatch. Axis units vary by predictor.

Numerical predictor	Train mean	Train SD	IQR outliers
duration	20.77	11.82	50
credit amount	3189.59	2673.55	58
installment commitment	2.94	1.12	0
residence since	2.83	1.11	0
age	35.32	11.05	25
existing credits	1.41	0.57	4
num dependents	1.16	0.37	128

Credit amounts and durations have visibly uneven scales and tails. The interquartile-range rule counts potential outliers descriptively; it does not establish erroneous records. These observations are retained because removing legitimate high-credit cases would change the task. Full numerical summaries and categorical counts are saved in q2_numeric_description.csv and q2_eda.json.

3.3 Preprocessing without leakage

A single stratified split with seed 42 is created before fitting any transform: 800 development records (560 good, 240 bad) and 200 test records (140 good, 60 bad). The identical split applies to all models. Five fixed stratified folds divide development records into 640 training and 160 validation records.

Feature type	Pipeline and rationale
Seven numerical predictors	Median imputation followed by StandardScaler. Each fold learns its own training statistics. Standardization supports distance, margin and linear methods.
Thirteen categorical predictors	Most-frequent imputation and dense one-hot encoding. Categories are learned only from fold training rows; unseen categories map to an all-zero block.
Target	good=0 and bad=1; this fixed semantic mapping is not a learned preprocessing step.
Clean-data finding	Imputers are included to define a robust pipeline but no values are actually imputed in the verified raw dataset.

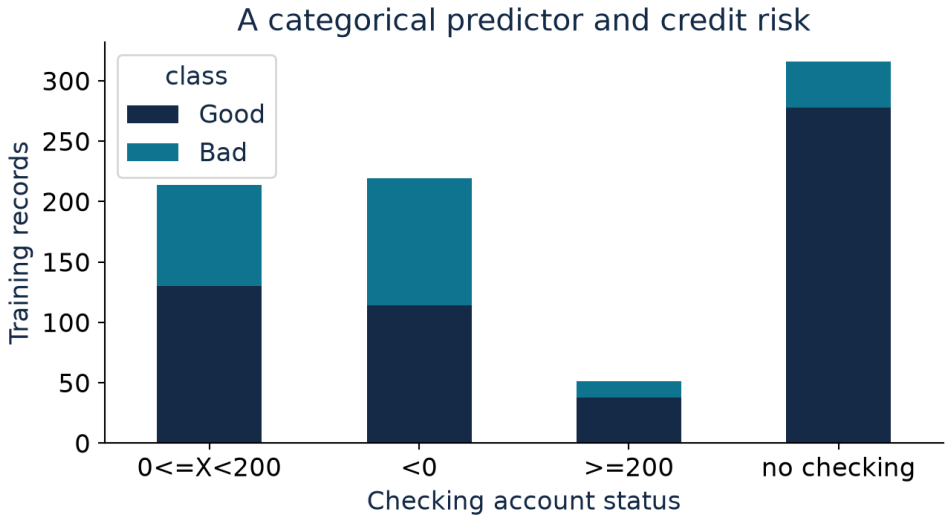


Figure 6. Development checking-account categories and credit risk counts. Association is descriptive, not causal.

The full development transform produces 61 columns. Tree thresholds are insensitive to positive affine rescaling of numeric predictors, so sharing the numeric scaler does not advantage trees. Dense one-hot input is manageable at this size; Gaussian Naive Bayes remains an approximate baseline because nominal indicator variables do not satisfy its Gaussian independence assumptions.

3.4 Model comparison protocol

Macro-F1 is chosen before viewing test results: compute each class F1 and average them equally. It gives the minority bad-credit class equal weight, unlike accuracy on a 70/30 split. Balanced accuracy and class-specific recall provide complementary views. This is an educational metric choice, not an implementation of the dataset cost matrix.

Classifier	Predefined main settings
Dummy majority	Most frequent label; reported separately as baseline.
Logistic Regression	C=1, max_iter=3000
KNN	k=5, uniform Euclidean neighbors
Gaussian Naive Bayes	Default variance smoothing
Decision Tree	Unrestricted depth, default leaf size
Random Forest / Extra Trees	150 trees each, n_jobs=1
Linear SVM	C=1, max_iter=20000
RBF SVM	C=1, gamma=scale
Gradient Boosting	100 stages, defaults otherwise
HistGradientBoosting	100 iterations, defaults otherwise
AdaBoost	100 estimators, learning rate 0.5
LDA	SVD solver; handles redundant one-hot directions

All stochastic estimators use seed 42. Complete estimator settings are serialized in q2_selection.json; unspecified settings are pinned by the recorded sklearn version. QDA is excluded because full one-hot indicators yield singular class covariance. MLP is omitted to bound runtime while retaining 12 meaningful learning algorithms.

Each estimator is a Pipeline containing its own ColumnTransformer. cross_validate clones and fits that entire pipeline inside each fold. Errors raise immediately and warnings remain in logs. CV uses the same folds, n_jobs=1 and five validation scores per candidate. The highest mean macro-F1 among predictive models wins, with alphabetical tie-break.

The selection file is written before any held-out prediction. The capacity studies also use development data only. Every predefined benchmark model is then fitted on development data and evaluated once on test data; the table does not trigger tuning.

3.5 Cross-validation and test comparison

The table is ordered by training-only mean macro-F1. The standard deviation is the sample SD across five folds (ddof=1); it is not a confidence interval and the folds are not independent experiments.

Model	CV F1 ± SD	Test accuracy	Test macro-F1
HistGradientBoosting	0.684 ± 0.040	0.755	0.687
Extra Trees	0.663 ± 0.058	0.765	0.693
Linear SVM	0.662 ± 0.071	0.775	0.722
LDA	0.662 ± 0.065	0.780	0.727
Logistic Regression	0.658 ± 0.072	0.780	0.721
RBF SVM	0.650 ± 0.051	0.790	0.720
Gradient Boosting	0.640 ± 0.035	0.795	0.741
Random Forest	0.639 ± 0.068	0.760	0.676
KNN	0.632 ± 0.048	0.725	0.626
AdaBoost	0.623 ± 0.035	0.760	0.671
Gaussian Naive Bayes	0.621 ± 0.129	0.695	0.664
Decision Tree	0.613 ± 0.015	0.620	0.552
Dummy majority	0.412 ± 0.000	0.700	0.412

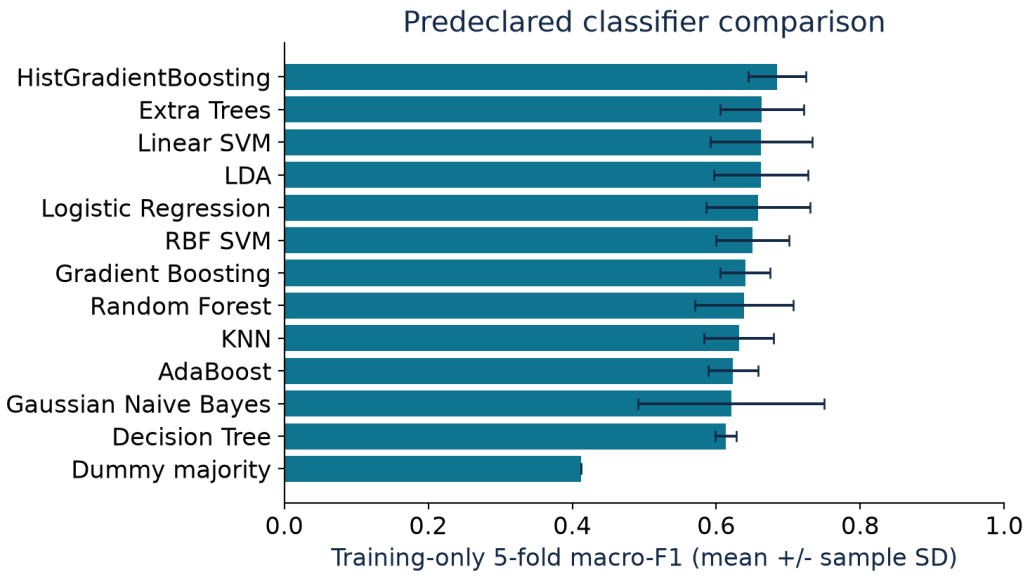


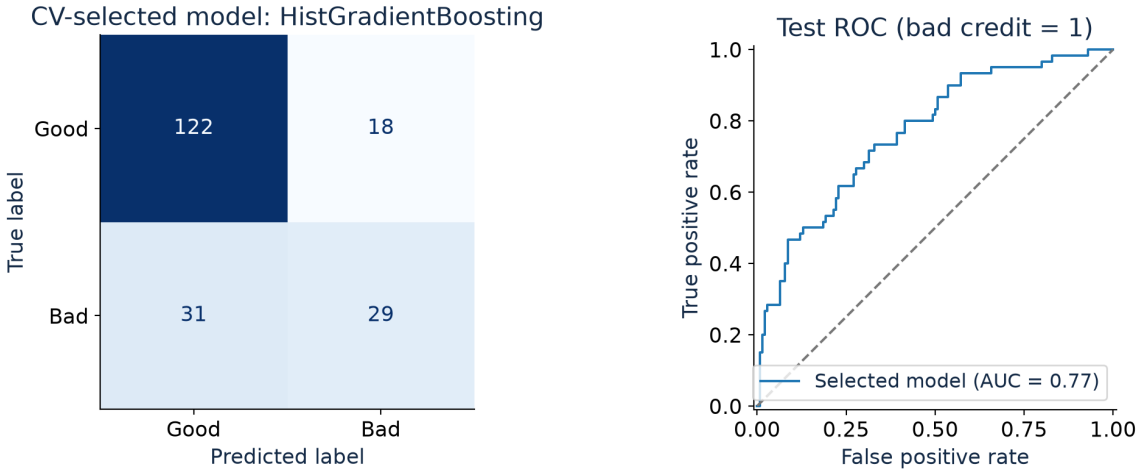
Figure 7. Mean development CV macro-F1 with sample SD. The zero-based axis avoids magnifying small differences.

Results are specific to this split, preprocessing and predefined settings. Close means relative to fold variability do not establish statistically significant differences. The majority baseline has 0.700 test accuracy but only 0.412 macro-F1, illustrating why accuracy alone can reward ignoring the minority class.

3.6 Selected model and generalization

The validation rule selects **HistGradientBoosting**, with CV macro-F1 0.6844 ± 0.0403. This estimator is selected before the test set is evaluated.

Held-out measurement	Value
Accuracy / macro-F1	0.755000 / 0.687410
Macro precision / macro recall	0.707203 / 0.677381
Bad-credit precision / recall	0.617021 / 0.483333
Balanced accuracy / ROC-AUC	0.677381 / 0.767857
Confusion matrix [TN, FP; FN, TP]	[[122, 18], [31, 29]]



Figures 8-9. Selected-model test confusion matrix and ROC. Bad credit is positive; ROC uses predicted scores rather than hard labels.

Validation estimates performance during selection; test estimates performance of the resulting fixed procedure on held-out observations. Some other candidates achieve higher test metrics on this split. Replacing the chosen model after reading those results would use the test as a selection signal.

The selected model has fold training macro-F1 1.0000 and a train/validation gap of 0.3156, so winning CV does not imply absence of overfitting. One changed prediction shifts test accuracy by 0.5 percentage points. Additional independent data would be required to validate stability.

CHAPTER 4. OVERFITTING ANALYSIS AND MITIGATION

4.1 Overfitting across model families

Overfitting is diagnosed through a high training score and weaker unseen validation score. The comparison uses fold training and validation on the same fits, avoiding a misleading comparison of unrelated training sizes. A gap is evidence of limited generalization; it does not by itself prove which mechanism caused the gap.

Model	Fold train F1	CV F1	Mean gap
HistGradientBoosting	1.000	0.684	0.316
Extra Trees	1.000	0.663	0.337
Linear SVM	0.733	0.662	0.070
LDA	0.727	0.662	0.066
Logistic Regression	0.728	0.658	0.069
RBF SVM	0.824	0.650	0.174
Gradient Boosting	0.911	0.640	0.271
Random Forest	1.000	0.639	0.361
KNN	0.741	0.632	0.110
AdaBoost	0.661	0.623	0.038
Gaussian Naive Bayes	0.661	0.621	0.040
Decision Tree	1.000	0.613	0.387
Dummy majority	0.412	0.412	0.000

Flexible trees attain perfect fold training scores with substantial validation gaps. The majority baseline has no gap yet underfits the minority. Both predictive quality and generalization matter.

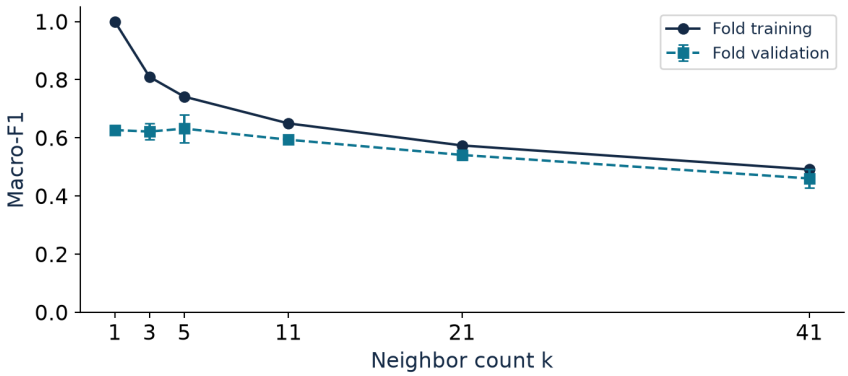


Figure 10. KNN complexity curve on the fixed development folds. Larger k smooths local predictions; vertical bars are sample SD. Tree and SVM configurations are detailed in Section 15.

All 20 settings use the identical five folds. No test score appears in capacity selection. These studies diagnose changes in complexity; they are not an exhaustive hyperparameter search.

4.2 Empirical mitigation and trade-offs

Decision Tree

Configuration	Fold train F1	CV F1	Gap
depth=None,leaf=1	1.0000	0.6128	0.3872
depth=None,leaf=15	0.7473	0.6594	0.0879

Best tested validation change: +0.0465; gap change: -0.2992. Full-fit training scores and every candidate configuration are retained in q2_studies.csv.

KNN

Configuration	Fold train F1	CV F1	Gap
k=1	1.0000	0.6265	0.3735
k=5	0.7415	0.6316	0.1099

Best tested validation change: +0.0050; gap change: -0.2635. Full-fit training scores and every candidate configuration are retained in q2_studies.csv.

RBF SVM

Configuration	Fold train F1	CV F1	Gap
C=100,gamma=1.0	1.0000	0.4201	0.5799
C=10.0,gamma=0.1	0.9996	0.6688	0.3308

Best tested validation change: +0.2487; gap change: -0.2491. Full-fit training scores and every candidate configuration are retained in q2_studies.csv.

Tree pre-pruning via minimum leaf size reduces tiny partitions and increases validation macro-F1 here. It is capacity restriction during construction, not iterative early stopping. With KNN, increasing k from 1 to 5 reduces local memorization, but the small CV gain should not be overstated.

The deliberately extreme RBF SVM (C=100, gamma=1) generalizes poorly. A less localized kernel improves CV substantially, though the selected C=10, gamma=0.1 configuration still has a large gap. Regularization can reduce a problem without eliminating it.

Not every stronger restriction improves validation. Some shallow trees and very large neighbor counts lose predictive capacity. All negative or weaker candidate outcomes remain visible in the study CSV and curve, rather than reporting only the best pair.

The best study scores reuse folds used to select settings and can therefore be optimistic. They support descriptive comparisons, not an unbiased estimate of tuning benefit. The final preferred model remains the benchmark CV winner; study settings are not promoted after examining test results.

4.3 Discussion and remaining limitations

Remaining family	Suggested control; not separately tuned here
Logistic Regression / Linear SVM	Reduce C to increase regularization; compare on training CV.
Random Forest / Extra Trees	Increase minimum leaf size, limit depth or features. More trees mainly reduce Monte Carlo variance, not necessarily capacity.
Gradient / histogram boosting	Reduce leaf complexity, stage count or learning rate; assess validation early stopping using training data only.
AdaBoost	Control weak-learner capacity and stage count; label noise can remain influential.
Gaussian Naive Bayes	Consider variance smoothing and assumption mismatch; low gaps may reflect underfitting.
LDA	Consider covariance shrinkage with an appropriate solver; evaluate rather than assume a gain.

Q1 demonstrates correct optimization on synthetic tasks, not universal regression or classification performance. The Logistic Regression fit is close to its unregularized reference but not exactly stationary; the measured tolerance and remaining difference are explicit.

Q2 uses a single stratified random split and a small historical dataset. Its imbalance, redundant indicators and possible demographic associations limit interpretation. The original cost matrix penalizes a bad customer classified as good more heavily [2]; macro-F1 is not a cost-sensitive objective. Neither fairness nor probability calibration is evaluated.

No statistical significance, deployment readiness or causal claim follows from these scores. Nested CV would be useful for a future tuned-procedure comparison but is not claimed here. Broader family-specific advice above is theoretical; only Tree, KNN and RBF SVM controls were experimentally compared.

Conclusion

Both scratch models use derived NumPy gradients and full-batch updates with independent numerical checks. Official OpenML provenance is verified, preprocessing is fitted within training folds, and validation evidence controls model selection. The capacity studies show both the usefulness and limits of stronger regularization. The companion source, executed notebooks and verification records make the reported evidence inspectable.

REFERENCES

[1] OpenML. credit-g, dataset 31, version 1. Official metadata and raw download. https://www.openml.org/api/v1/json/data/31 and https://www.openml.org/d/31. Retrieved 9 October 2026.

[2] Hofmann, H. (1994). Statlog (German Credit Data). UCI Machine Learning Repository. DOI: 10.24432/C5NC77. https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data. CC BY 4.0; accessed 9 October 2026.

[3] scikit-learn developers. Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html. Accessed 9 October 2026.

[4] scikit-learn developers. Pipelines and composite estimators; ColumnTransformer for heterogeneous data. https://scikit-learn.org/stable/modules/compose.html. Accessed 9 October 2026.

[5] scikit-learn developers. Linear models; Linear and Logistic Regression. https://scikit-learn.org/stable/modules/linear_model.html. Accessed 9 October 2026.

[6] scikit-learn developers. Metrics and scoring: quantifying prediction quality. https://scikit-learn.org/stable/modules/model_evaluation.html. Accessed 9 October 2026.

Attribution and evidence

Dataset attribution identifies the original creator and originating repository, while the experiment uses the specific OpenML ARFF distribution. Source URLs and metadata are retained so the data identity can be checked independently. No equivalence claim is made between credit-g and the previous Iris CSV.

The loss derivations in Sections 2 and 5 are shown in full. References provide established model and workflow context; they are not substitutes for the submitted calculations or experiment execution. Figures are generated from the supplied raw data and measured optimization histories.

DejaVu fonts are bundled with their license to support offline PDF reconstruction. No external stock imagery or generated illustrative data appears in this report.

APPENDIX. REPRODUCIBILITY AND VERIFICATION

Artifact	Role
src/q1_models.py	Scratch algorithms, synthetic data, independent reference checks and results.
src/data.py	Official retrieval and raw artifact verification.
src/q2_classification.py	Development EDA, fold-safe pipelines, comparison and capacity experiments.
notebooks/	Three teaching notebooks executed from fresh kernels.
data/raw and data/metadata	ARFF, official response, provenance, split indices and CV folds.
results/	Metrics, per-fold scores, capacity settings, predictions and execution logs.
tests/ and verification/	Mathematical, data, leakage, determinism, notebook, report and archive checks.
report/report_source/	Editable metadata, generated manuscript, font licenses and numerical trace.

Reproduction order

Use Python 3.14 with the pinned requirements, from the extracted project root. Verify the cached dataset, run Q1, run Q2, execute the three notebooks, run unittest, then rebuild the PDF. Fish-compatible commands are in README.md. BLAS/OpenMP and estimator parallelism are capped at one thread.

Expected evidence

Q1 tolerances are assertions in the experiment and tests. Q2 test metrics are recomputed from stored predictions; fold means, SD and gaps are recomputed from fold scores. Fold-specific imputer, scaler and category statistics are tested directly. Report numeric claims read generated JSON/CSV files; rendered PDF QA is documented separately.

Every member must submit

The lecturer requires every group member to submit both the PDF and code. Confirm the student IDs on the cover, review the study guide, and understand the derivations and model-selection protocol before submission. No class section or institution has been invented.

Verification reports retain genuine failures and limitations. Passing finite-difference checks verifies tested code numerically, not all possible inputs. Readiness is stated in verification/requirements_traceability.md after all feasible execution gates.